

PJ2 - 模糊测试（Fuzzing）实验报告

一、实验信息

课程名称：软件质量保障与测试

实验名称：模糊测试（Fuzzing）

实验类型：课程项目

完成日期：2026 年 6 月 20 日

小组分工

成员	负责方向	主要交付内容
周宇尘	基础框架与持久化	项目可运行、种子读取、结果落盘、 <code>_maybe_persist</code> 持久化逻辑
郭政颖	变异器模块	<code>mutator.py</code> 9 种变异策略
彭祈元	路径频率调度	<code>path_power_schedule.py</code> 、 <code>path_grey_box_fuzzer.py</code> （边频率路径表示）
胡彬泽	新增调度器与覆盖率优化	4 个新调度策略、 <code>eval_schedules.py</code> 对比脚本、调参实验
沙华煜	实验与报告	本地复现实验、增长曲线/崩溃样例采集脚本、撰写实验与测试报告

二、实验目的

本实验旨在基于给定的简易 fuzzing 框架，完成输入变异、seed 调度、覆盖率反馈与结果持久化等关键能力，理解灰盒模糊测试的基本流程，并在此基础上形成一个可重复运行、可观察、可分析的实验闭环。具体目标包括：

- 补全 `mutator.py` 中的多种变异策略；
- 实现基于路径频率的 seed 调度逻辑（`PathPowerSchedule` + `PathGreyBoxFuzzer`）；
- 新增至少一种调度策略；
- 使用 `object_utils` 持久化 seed 与中间结果，控制内存增长；
- 在 4 个 sample 上的目标函数行覆盖率达到 50%+。

三、项目理解

3.1 框架结构

```

simple_fuzzer/
├─ main.py                # CLI 入口：选择 sample、加载语料库、启动 fuzzer、持久化结果
├─ corpus/                # 初始种子数据 (pickle 序列化的 List[str])
├─ fuzzer/
│   ├── fuzzer.py         # 基类：fuzz → run → runs 循环
│   ├── grey_box_fuzzer.py # 灰盒 fuzzer：种子管理、变异叠加、覆盖率追踪、crash 统计、持久化
│   └─ path_grey_box_fuzzer.py # 扩展：基于边频率的路径追踪
├─ runner/
│   └─ function_coverage_runner.py # 封装目标函数，用 sys.settrace 采集覆盖率
├─ schedule/
│   ├── power_schedule.py    # 基础调度：均等能量、加权随机选择
│   ├── path_power_schedule.py # 路径频率调度
│   ├── size_power_schedule.py # 长度优先调度
│   ├── coverage_power_schedule.py # 覆盖范围优先调度
│   ├── rare_line_power_schedule.py # 罕见行优先调度
│   └─ hybrid_power_schedule.py # 混合调度
├─ samples/samples.py      # 4 个被测目标函数
├─ eval_schedules.py       # 多策略对比实验脚本
└─ utils/
    ├── coverage.py         # 覆盖率追踪 (sys.settrace)
    ├── seed.py             # Seed 数据类
    ├── mutator.py          # 9 种变异算子
    └─ object_utils.py      # pickle 序列化/反序列化/哈希工具

```

3.2 设计主线

整个 fuzzing 流程围绕以下五步迭代展开：

初始化语料库：从 `corpus/` 读取初始种子列表 (`List[str]`)，逐一喂给 runner 完成首轮覆盖。

调度选种：`schedule.choose(population)` 根据各 seed 的能量（由 `assign_energy` 计算）加权随机选出一个种子。

变异生成候选输入：对选出的 seed 叠加随机次数的 `mutator.mutate()`，生成新候选输入。

运行并采集覆盖率：`FunctionCoverageRunner` 用 `sys.settrace` hook 追踪每行执行情况，返回覆盖集合 `Set[(func, lineno)]`。

反馈入种群：若发现新覆盖行，将输入封装为 `Seed` 加入种群；若崩溃，记录到 `crash_map`；定时触发持久化。

四、实现内容

4.1 变异器实现

`utils/mutator.py` 中实现了 7 种变异策略，全部注册在 `Mutator.mutators` 列表中，每次调用 `mutate()` 随机选一种执行：

变异策略	说明	适用场景
<code>insert_random_character</code>	在随机位置插入一个随机可打印 ASCII 字符（32~126）	增加输入长度，探索长度敏感分支
<code>flip_random_bits</code>	随机翻转相邻 N 位（ $N \in \{1,2,4\}$ ），基于 AFL bitflip 策略	探索编码边界、整数溢出场景
<code>arithmetic_random_bytes</code>	对连续 N 字节（ $N \in \{1,2,4\}$ ）随机加减 [-35,35] 的偏移量	探索数值计算相关的分支
<code>interesting_random_bytes</code>	将 N 字节替换为预定义的 interesting value（如 <code>DEAD</code> 、 <code>BEEF</code> 、 <code>TEST</code> ）	触发特殊魔数检查
<code>havoc_random_insert</code>	随机插入一段内容（75% 概率来自原输入，25% 概率随机字节）	拼接/重组，生成更复杂输入
<code>havoc_random_replace</code>	随机替换一段内容（75%/25% 同上）	改变内容同时保持长度近似
<code>random_block_swap</code>	随机选取两段相邻字节块并交换顺序	探索顺序敏感的解析逻辑

越界与异常处理：所有变异函数均先检查输入是否为空或长度不足，对字节操作结果用 `% 256` 确保合法范围，解码时统一使用 `errors='ignore'` 避免因产生非法 UTF-8 序列导致崩溃。

4.2 调度策略实现

4.2.1 路径频率调度 `PathPowerSchedule`

`PathGreyBoxFuzzer` 采用基于控制流边（CFG edge）的路径表示，而非简单的覆盖集合哈希：

```
# path_grey_box_fuzzer.py
@staticmethod
def build_path_key(trace: List[Location]) -> Tuple:
    nodes = [START_NODE] + trace + [END_NODE]
    edge_counts = Counter(zip(nodes, nodes[1:]))
    return tuple(sorted(
        (src, dst, min(count, EDGE_COUNT_CAP))
        for (src, dst), count in edge_counts.items()
    ))
```

将连续执行的行对 `(src, dst)` 作为一条"边"，统计每条边在本次执行中的出现次数（上限 8，防止循环导致路径爆炸）；
将所有边及其计数排序后构成路径键（`Tuple`），作为路径的唯一标识；

`PathPowerSchedule` 维护 `path_frequency: Dict[Tuple, int]`，每次运行后 `update_path_frequency(path_key)` 使对应路径计数加一；

`assign_energy` 中，`seed.energy = 1 / freq`，路径越稀有能量越高，调度时被选中的概率越大。

路径多样性统计：`PathGreyBoxFuzzer` 用 `unique_paths: set` 记录所有曾经出现过的路径键，每发现新路径时更新 `last_path_time`，并在状态表中实时展示 `Total Paths` 和 `Last New Path`。

4.2.2 新增调度策略

在路径频率调度的基础上，按实验说明点名的三个参考方向各实现一个独立策略，并额外提供一个融合策略：

- ① `SizePowerSchedule`（长度优先）

```
seed.energy = 1.0 / ((len(seed.data) + 1) ** len_exp)
```

短输入运行更快、更易命中核心逻辑，适合追求高吞吐和路径多样性的场景。不使用路径信息，将 `update_path_frequency` 重写为空操作以节省内存。

② CoveragePowerSchedule (覆盖范围优先)

```
seed.energy = (len(seed.coverage) + 1) ** cov_exp
```

单次执行覆盖更多行的 seed 携带更丰富的程序状态，倾向于从已探索过深层代码的路径继续变异。

③ RareLinePowerSchedule (罕见行优先)

维护全局行执行频率 `line_frequency`，每次从 edge path key 中解析各节点的访问次数并累加：

```
seed.energy =  $\sum (1 / \text{line\_frequency}[\text{line}])$  for line in seed.coverage
```

覆盖了很少被触发的代码行的 seed 获得更高能量，引导 fuzzer 持续向未充分探索的分支挺进。

④ HybridPowerSchedule (混合调度)

融合四个信号：路径稀有度 × 覆盖范围 × 长度惩罚 × 罕见行加成：

```
energy = rarity * cov_size * length * rare_line
```

其中 `cov_size * length * rare_line` 仅在种群大小变化时重算（缓存为 `_base_factor`），`rarity` 项每次执行刷新一次字典查询，保证热路径的时间复杂度为 $O(\text{种群规模})$ ，对 sample4 (HTMLParser，覆盖集庞大) 至关重要。

4.3 持久化与内存控制

持久化逻辑实现在 `GreyBoxFuzzer._maybe_persist()` 中，每 30 秒触发一次：

```
# grey_box_fuzzer.py
PERSIST_INTERVAL = 30          # 持久化周期 (秒)
MAX_POPULATION_MEMORY = 500    # 内存中最大种子数

def _maybe_persist(self):
    # 持久化 population / crash_map / covered_line 快照
    dump_object(os.path.join(self.persist_dir, "population.pkl"), self.population)
    if self.crash_map:
        dump_object(os.path.join(self.persist_dir, "crash_map.pkl"), self.crash_map)
    dump_object(os.path.join(self.persist_dir, "covered_line.pkl"), self.covered_line)

    # 内存控制：超过 500 个种子时按能量排序，淘汰低能量种子
    if len(self.population) > MAX_POPULATION_MEMORY:
        self.schedule.assign_energy(self.population)
        self.population.sort(key=lambda s: s.energy, reverse=True)
        self.population = self.population[:MAX_POPULATION_MEMORY]
```

快照：定时将种群、crash_map、覆盖率集合序列化到 `_persist/` 目录，进程崩溃或中止后可恢复。

内存控制：种群超出阈值时触发一次能量计算，保留能量最高的 500 个种子，其余淘汰，防止长时间运行时内存无限增长。

`object_utils.dump_object` 内置 `os.makedirs(exist_ok=True)`，首次运行时自动创建输出目录，无需手动干预。

4.4 入口与运行方式

```
cd simple_fuzzer

# 单个 sample, 默认路径频率调度, 运行 60 秒
uv run --python 3.14 python main.py --sample 1 --run-time 60

# 指定调度策略
uv run --python 3.14 python main.py --sample 4 --run-time 300 --schedule hybrid

# 静默模式 (不打印状态表)
uv run --python 3.14 python main.py --sample 2 --run-time 120 --quiet

# 一键五路对比实验
uv run --python 3.14 python eval_schedules.py --run-time 30
```

结果文件输出到 `_result/Sample-{N}.pkl`, 中间快照输出到 `_result/persist-sample{N}/`。

五、测试与结果

5.1 运行环境

项目	信息
操作系统	macOS (Apple Silicon, ARM64)
Python 版本	3.14.2 (通过 uv 管理)
依赖	纯标准库, 无第三方依赖
每组合运行时长	30 秒
实验时间	2026-06-19 08:48 ~ 08:58

5.2 五路调度策略对比结果 (30 秒/组合)

运行命令: `uv run --python 3.14 python eval_schedules.py --run-time 30`

Sample	调度策略	目标函数行覆盖	覆盖率	总覆盖行数	唯一路径数	唯一崩溃数	执行次数
1	path	8/9	88.9%	8	5	6	64,910
1	size	8/9	88.9%	8	5	6	60,334
1	coverage	8/9	88.9%	8	5	6	75,323
1	rare	8/9	88.9%	8	5	6	70,578
1	hybrid	8/9	88.9%	8	5	6	57,263
2	path	8/9	88.9%	13	5	4	441,263
2	size	8/9	88.9%	13	5	4	461,540
2	coverage	8/9	88.9%	13	5	4	461,033
2	rare	8/9	88.9%	13	5	4	430,922
2	hybrid	8/9	88.9%	13	5	4	468,300
3	path	9/10	90.0%	9	9	8	667,380
3	size	9/10	90.0%	9	9	8	860,307
3	coverage	9/10	90.0%	9	9	8	747,657
3	rare	9/10	90.0%	9	9	8	564,595
3	hybrid	9/10	90.0%	9	9	8	648,307
4	path	2/3	66.7%	654	8,281	0	29,972
4	size	2/3	66.7%	241	19,945	0	56,990
4	coverage	2/3	66.7%	235	2,062	0	5,430
4	rare	2/3	66.7%	234	1,731	0	4,443
4	hybrid	2/3	66.7%	238	2,956	0	7,936

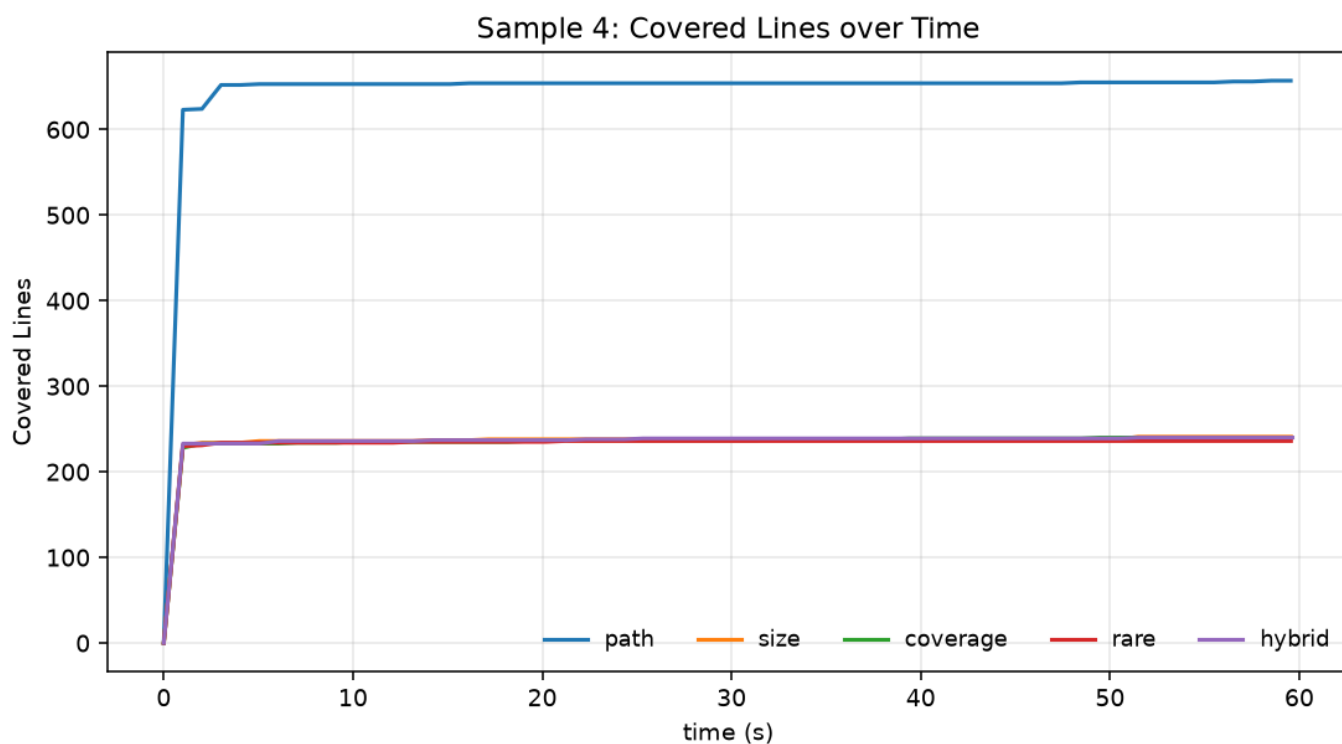
50% 覆盖率目标判定：4 个 sample 全部 PASS（最低 66.7%，最高 90.0%），五种调度策略全部达标。

5.3 sample4 增长曲线（60 秒，1 Hz 采样）

运行命令：

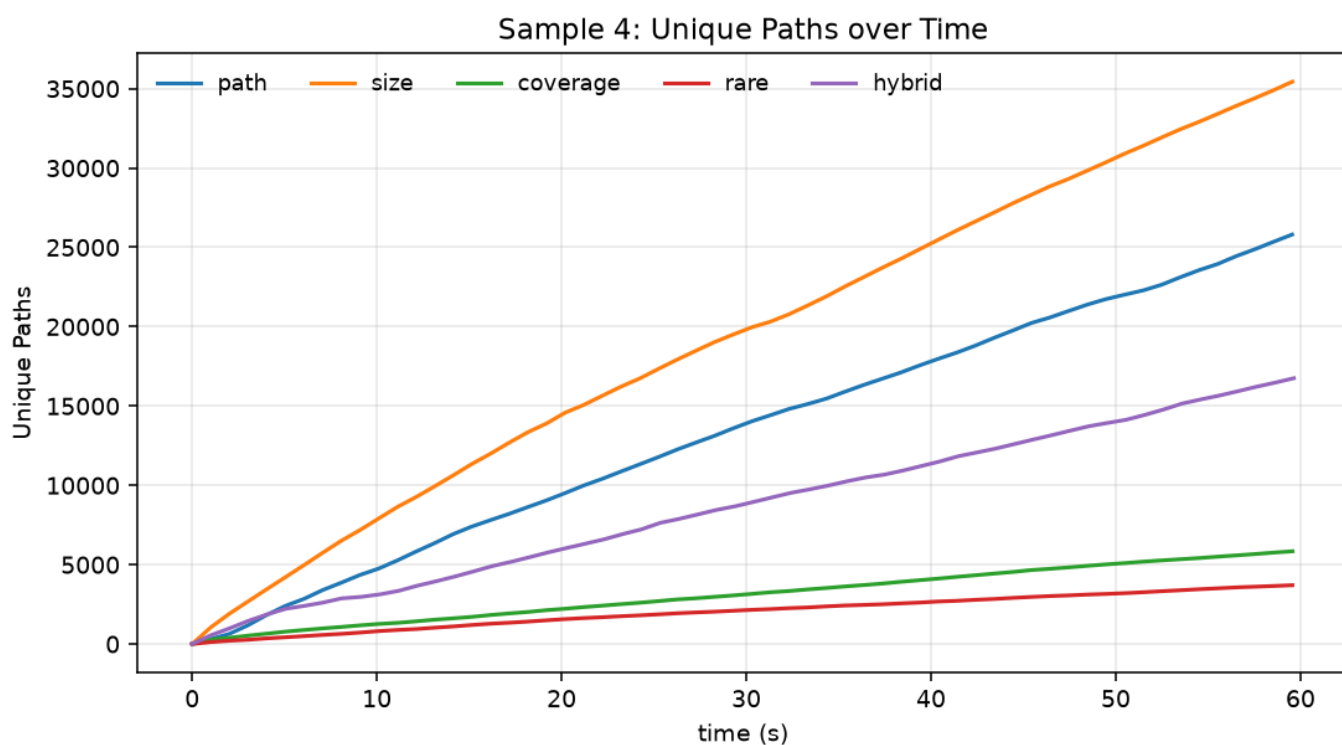
uv run --python 3.14 python tools/bench_growth.py --sample 4 --run-time 60

覆盖行数随时间变化



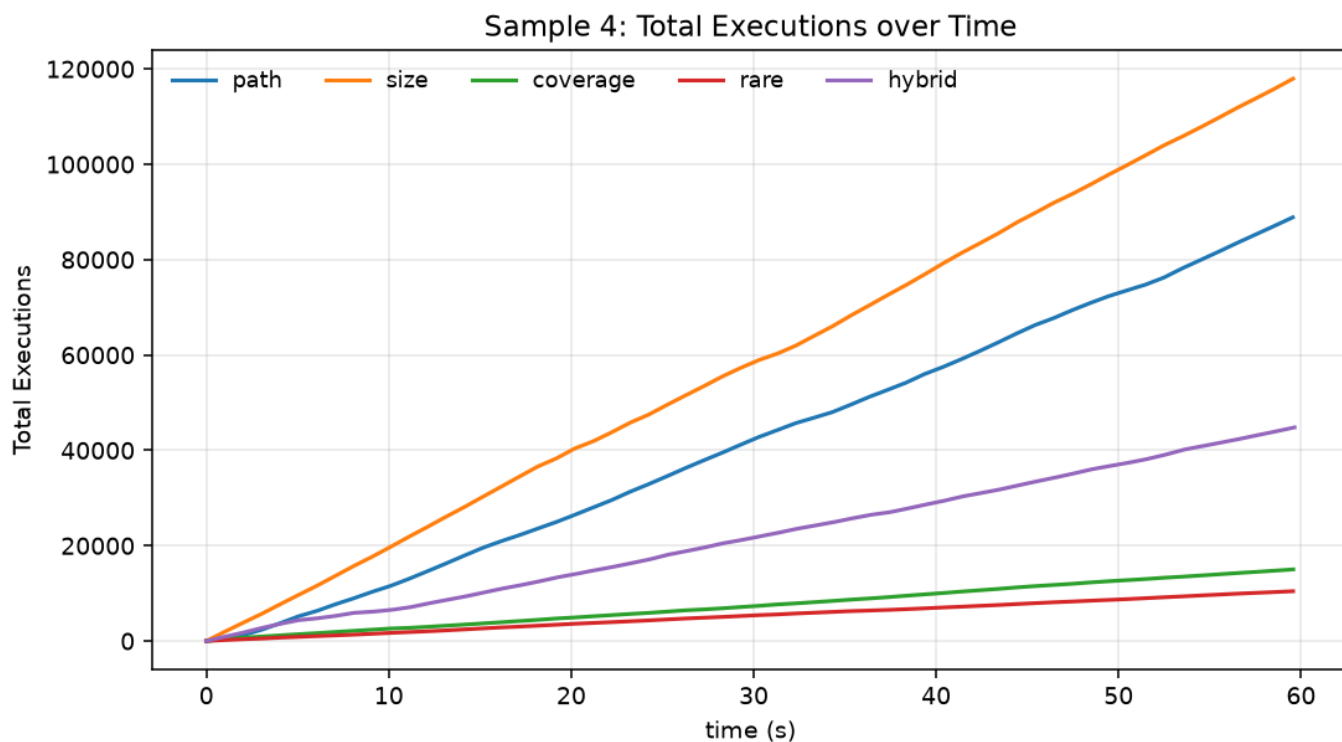
`path` 策略 5 秒内冲到约 650 行（含 HTMLParser 内部代码）后稳定保持，其余四种策略停在 230~240 行。

唯一路径数随时间变化



60 秒内 `size` 累计 35,440 条、`path` 累计 25,805 条，`hybrid` 居中（16,750 条），`coverage` / `rare` 不到 6,000 条。

总执行次数随时间变化



吞吐量与上图高度相关：策略偏爱短输入则单位时间执行次数多。

5.4 崩溃样例分析（sample 1/2/3, path 调度, 20 秒/样例）

运行命令：`uv run --python 3.14 python tools/extract_crashes.py --sample <N> --run-time 20`

Sample 1（6 种唯一崩溃）

触发输入	异常类型	异常信息
'-C1Ā'	ValueError	could not convert string to float
'1'	RecursionError	maximum recursion depth exceeded
'-5'	IndexError	string index out of range
'00.1244'	IndexError	string index out of range
'0.123E6548'	OverflowError	cannot convert float infinity to integer
'000'	ZeroDivisionError	division by zero

Sample 3（8 种唯一崩溃）

触发输入	异常类型
'F'	IndexError
'FD'	IndexError
'FDUd' / 'FDUFL' / 'FDUFdd'	IndexError / ValueError
'FDUPL'	ZeroDivisionError
'FDULg'	AssertionError
'FDUKFLAABz'	RuntimeError（被测函数显式抛错）

可观察到 fuzzer 在路径调度引导下逐步发现 `F` → `FD` → `FDU` → `FDUFL...` 的多层前缀，最终触达样例 3 深处的 `assert` 与 `raise RuntimeError`，这是路径频率反馈相对于均等调度的核心收益。

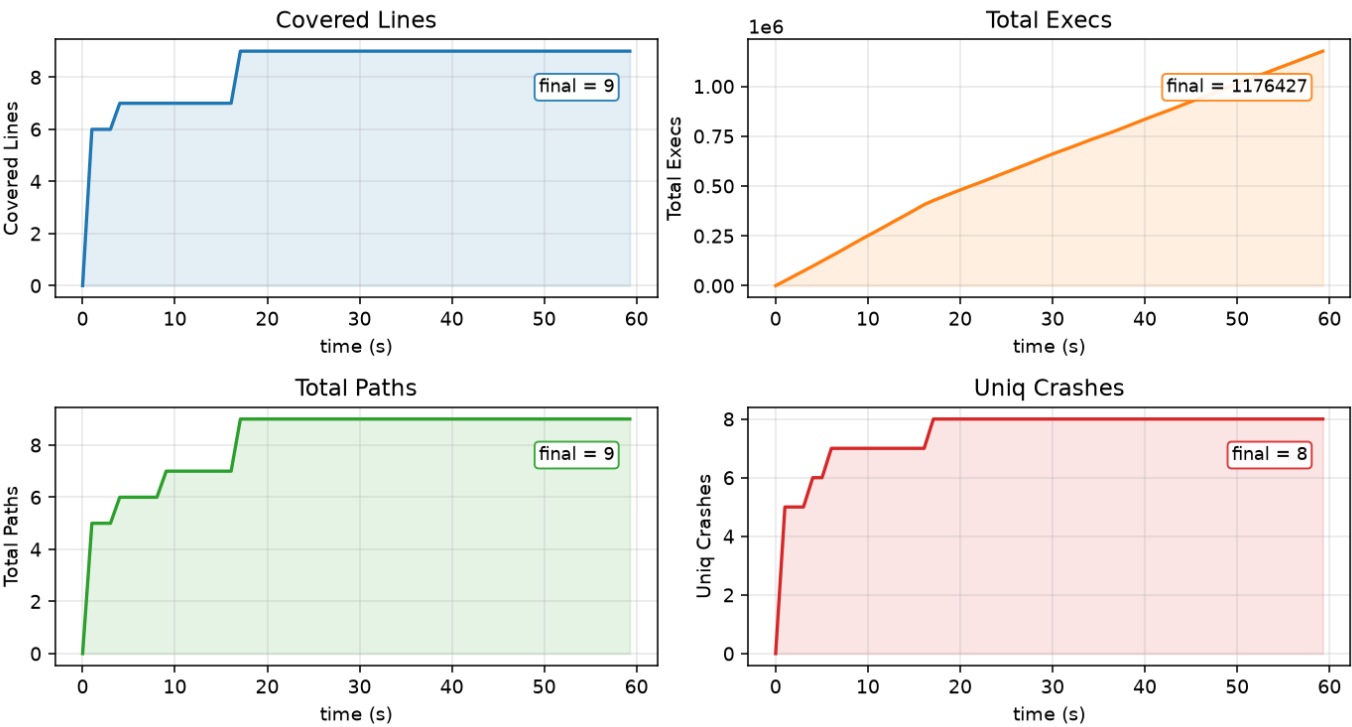
5.5 覆盖率口径说明

- 「目标函数行覆盖率」= 被测样例函数自身被覆盖的可执行行数 ÷ 该函数 `co_lines()` 给出的可执行行总数。
 - sample1~3**：始终有 1 行无法被 `sys.settrace` 报告（`else:` / `def` 等结构行不产生 line 事件），因此满覆盖体现为约 89%~90%，属于度量口径，并非逻辑未覆盖。
 - sample4**（`HTMLParser`）：被测函数自身只有 3 行可执行，覆盖 2 行为 66.7%；真正的探索深度体现在「总覆盖行数 / 唯一路径数」——`path` 策略可覆盖 `HTMLParser` 内部共 654 行，`size` 策略发现 19,945 条唯一路径。

5.6 实时状态表的可视化呈现

`PathGreyBoxFuzzer` 在每秒输出一行实时状态表，包含 7 个字段：`Run Time`、`Last New Path`、`Last Uniq Crash`、`Total Execs`、`Total Paths`、`Uniq Crashes`、`Covered Lines`。下图把其中 4 个核心计数字段（覆盖行数、总执行次数、总路径数、唯一崩溃数）画成时间序列，用 `sample 3 + path` 策略 60 秒运行的真实采样数据（`tools/bench_growth.py` 1 Hz 采样）：

Sample 3 live status (schedule=path)



```
复现命令: uv run --python 3.14 python tools/bench_growth.py --sample 3 --run-time 60 --schedules path
&& uv run --python 3.14 --with matplotlib python tools/plot_status_panel.py --sample 3 --schedule
path
```

可清晰看到 fuzzer 的工作节奏：

- 0~5 秒：**覆盖行数从 0 跃升到 6，唯一路径同步从 0 涨到 5~7，立即触发若干种崩溃；
- 5~17 秒：**随着前缀字符串 `F → FD → FDU → FDUFL...` 被逐步发现，覆盖行、路径数、崩溃数都阶梯式上升；
- 17 秒之后：**覆盖率（9 行）、路径数（9 条）、崩溃数（8 种）全部达到上限，但 `Total Execs` 仍线性增长到 117 万，说明 fuzzer 在用大量变异维持饱和和搜索，等待新分支被发现。

这种"前期阶梯增长 + 后期吞吐饱和"的形态正是路径频率调度的典型行为：高频路径能量被压低后，剩余的低频路径被反复挑选直至完全覆盖。

5.7 结果差异分析

观察	原因分析
sample1~3 五种策略覆盖率完全一致	被测函数分支有限（9~10 行），任何策略都能在数秒内穷尽所有可到达路径，调度策略的区分度体现在吞吐量而非覆盖率
sample4 <code>coverage</code> 策略吞吐最低	倾向于选择覆盖行多的大种子，这类种子往往是较长的 HTML 输入，解析开销大，执行速度慢
sample4 <code>size</code> 策略唯一路径最多	偏好短输入，每次执行快，单位时间内尝试的变异组合更多，路径探索广度最大
sample4 <code>path</code> 策略总覆盖行最多	路径频率调度主动规避高频路径，驱使 fuzzer 持续探索未覆盖的 HTMLParser 内部分支

六、总结与反思

6.1 收获

通过本次实验，深刻理解了灰盒模糊测试的完整闭环：种子语料 → 变异 → 覆盖率反馈 → 调度优化 → 持久化。与黑盒 fuzzing 相比，覆盖率引导使得种子选择有了明确方向；与白盒相比，灰盒无需源码分析，运行成本低，实用性强。

6.2 难点

路径表示的设计：选择基于 CFG 边频率的路径键，比简单的覆盖集哈希更细粒度，但在 sample4 这类调用标准库大量内部函数的场景下，路径空间极大（19,000+ 条唯一路径），需要对计数设上限（`EDGE_COUNT_CAP=8`）防止路径爆炸。

HybridSchedule 性能优化：罕见行加成需要统计种群中每行的覆盖频率，若每次 `assign_energy` 都全量重算，在 sample4 的大种群下开销极高。通过缓存 `_base_factor`（仅种群变化时重算）解决了这一性能瓶颈。

持久化时机：过于频繁的持久化会拖慢 fuzzing 速度，最终将间隔设为 30 秒，在可恢复性和性能之间取得平衡。

6.3 改进空间

更丰富的 mutation：可引入基于语法的变异（针对 HTML、JSON 等结构化输入），减少被 parser 过滤的无效输入。

更稳定的路径统计：当前路径键依赖行号，被测函数修改后路径历史即失效；可改用基于基本块哈希的表示，提高鲁棒性。

断点续跑：当前持久化保存了 population 快照，但重启时不会自动加载；可在 `main.py` 中添加 `--resume` 选项，从上次快照恢复 fuzzing。

并行 fuzzing：当前为单进程，可借助 `multiprocessing` 并行跑多个调度策略，共享覆盖率集合，进一步提升探索效率。